

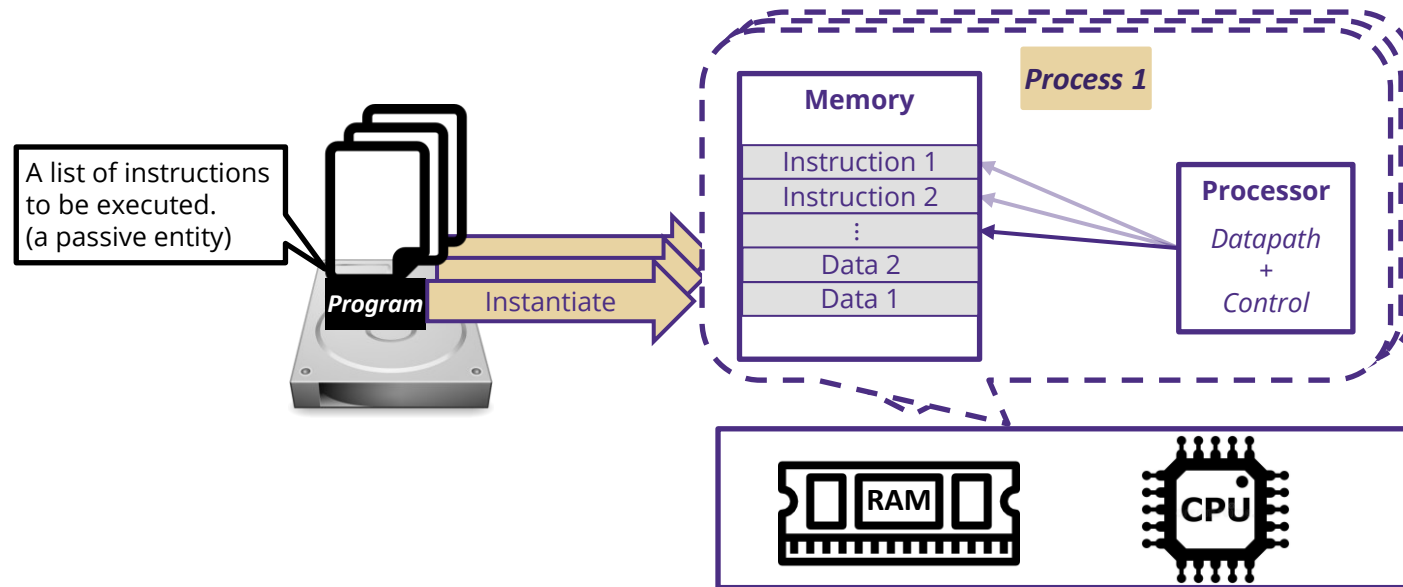
ADVANCED PROGRAMMING TECHNIQUES

LECTURE 3: Working with Processes in Linux

CONTENTS

- > Introduction
- > Linux process lifecycle
 - Process creation
 - Process termination
 - Loading a program into a child process
 - Monitoring a child process
- > Interprocess communication (IPC)
 - Signals
 - Pipes

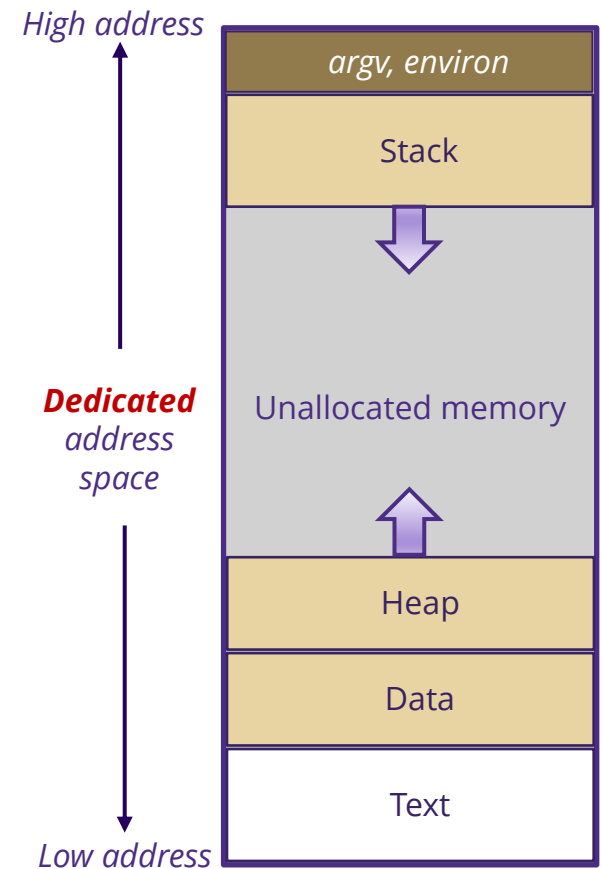
Introduction



- > A process is an instance of a program in execution, i.e. an **active** entity that holds the resources a program needs to execute.
 - Computer runs many processes simultaneously.
 - Process gives the running program a **state** via two key abstractions:
 - ✓ Logical control flow: the program seems to have exclusive use of the CPU
 - ✓ Private memory: the program seems to have exclusive use of memory.

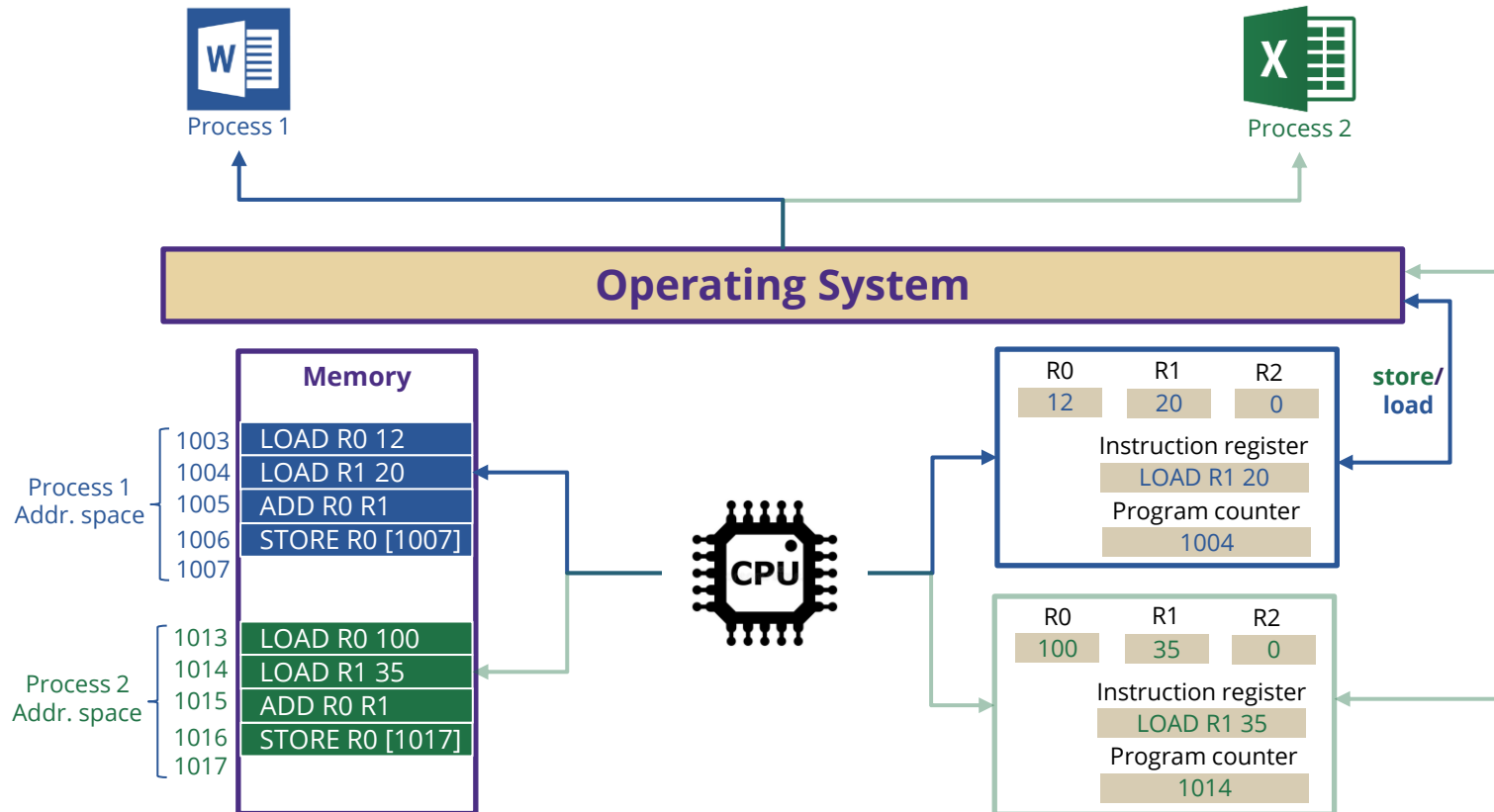
Process Memory

- > Text: holds the program code
 - Read only to prevent self-modification
 - May be shared with other processes
- > Data: holds global & static variables
 - Initialized
 - Uninitialized (technically initialized to 0)
- > Stack: holds function's local variables
 - Grows & shrinks as the program executes
- > Heap: holds dynamic data structures
 - `malloc()`, `calloc()`, `free()`, etc.



CPU States

- > Linux runs multiple processes concurrently by sharing CPU time
 - OS must store the current CPU state before switching to another process.



Process Context

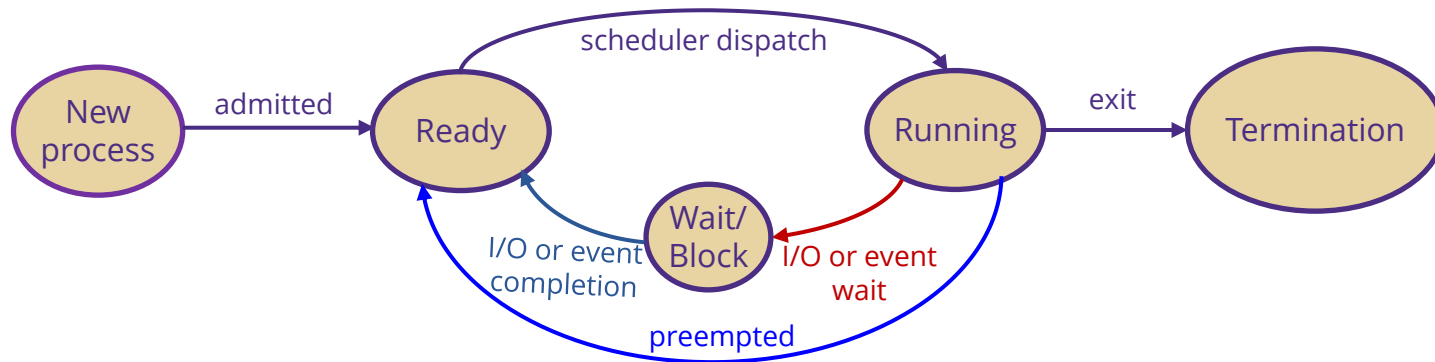
- > **Def.:** everything the OS needs to manage running processes.
 - **User context**, e.g., process memory, environment variables, open fd's, ...
 - **Kernel context**, e.g., process ID (PID), parent process ID (PPID), I/O buffers, ...
 - **Hardware context**, e.g., CPU registers, PC, ...



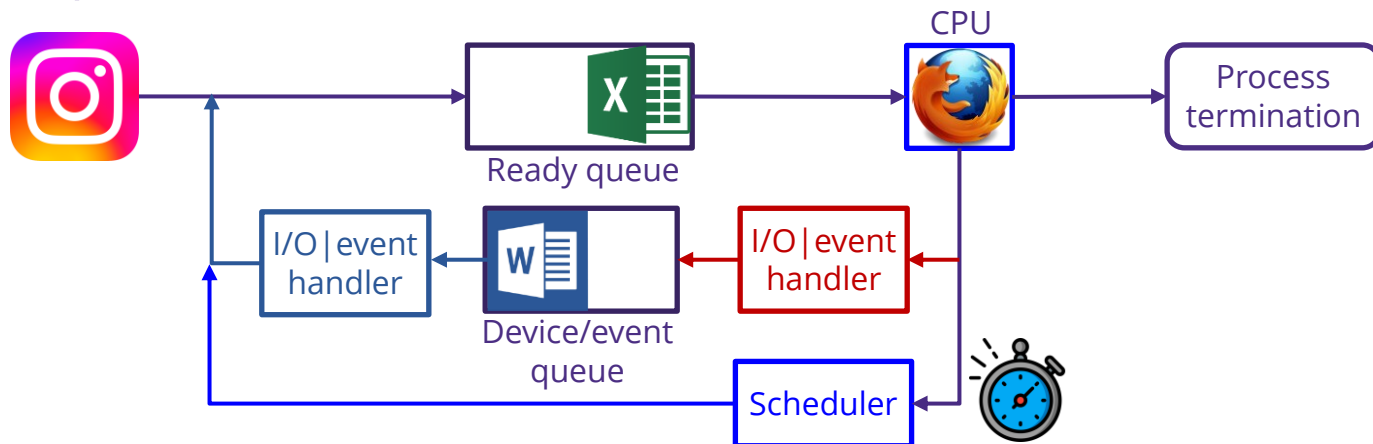
- > Process context switching occurs when
 - The OS switches between processes (preemptive multitasking)
 - A process blocks (waiting for I/O, sleeping)
 - A higher-priority process needs CPU time

Process Life Cycle

> Each process has 5 basic states:

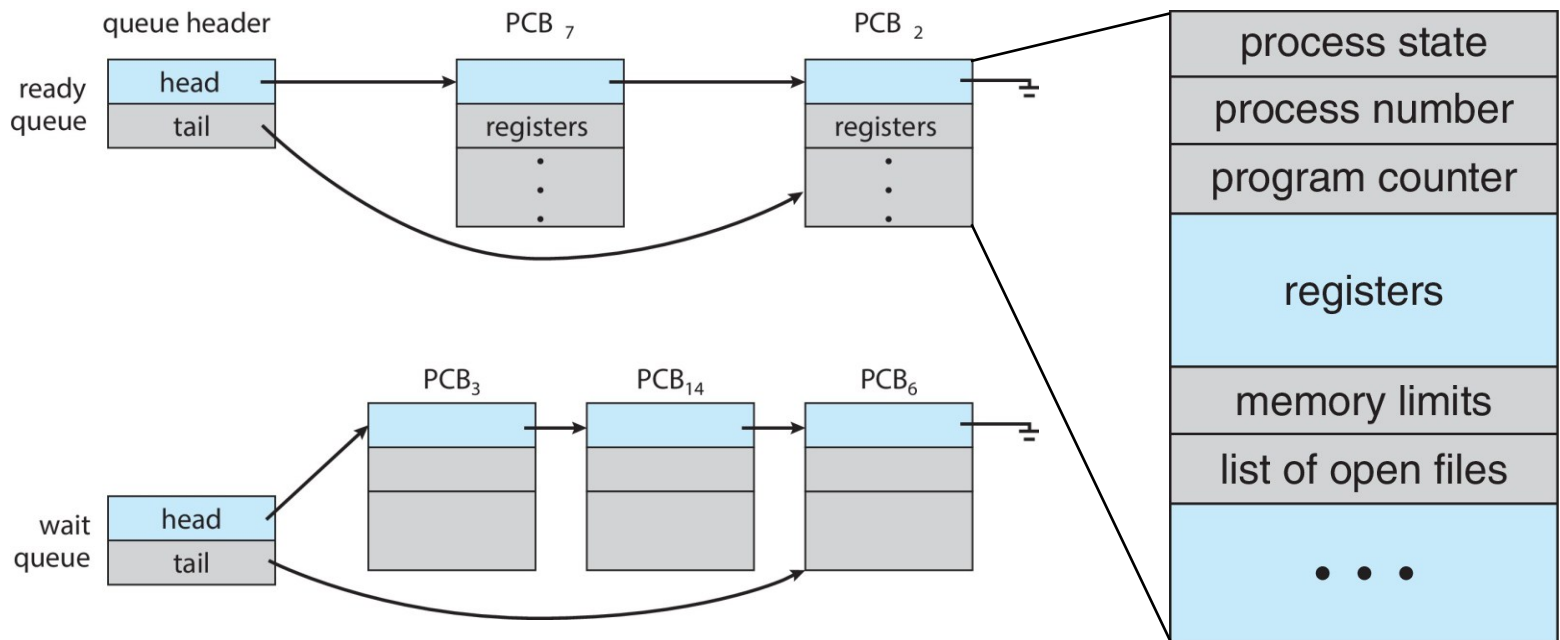


> Example:

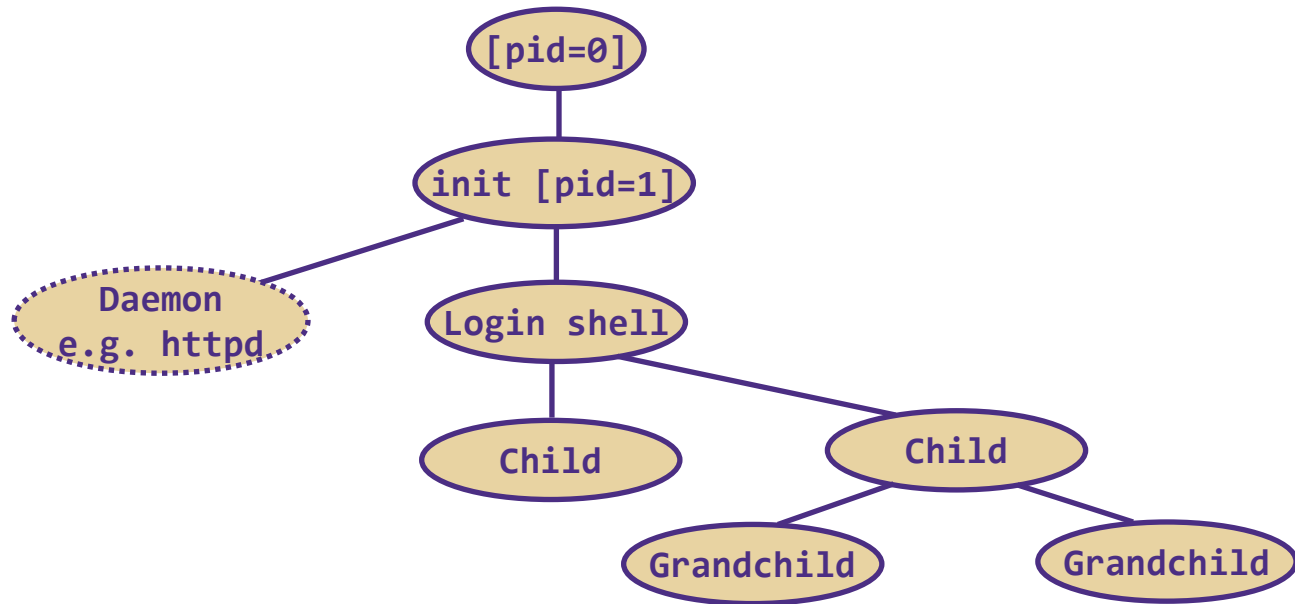


Process Control Block (PCB)

- > PCB = a data structure in the OS kernel containing information needed to manage a particular process.
 - Information in the PCB may vary from process to process.
 - Process queues can be formed by linking PCBs together.



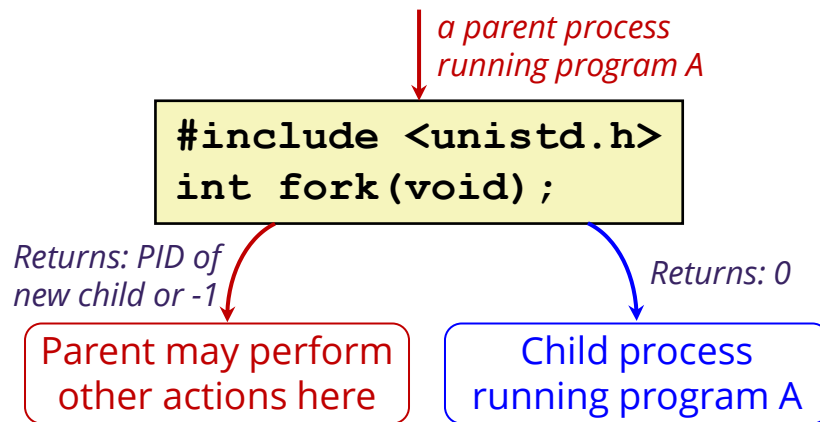
Process Creation



- > Creating multiple processes is a useful way of dividing a task
 - A *parent* process creates *children* processes, which, in turn create other processes, forming a *tree* of processes.
 - **Example:** a server process is listening for incoming client requests, and
 - ✓ upon receipt, creates a new child process to service the request;
 - ✓ the parent process, meanwhile, keeps listening for further requests.

Process Creation in Linux

- > A process is created from the parent process via `fork()`



- > Child is a near exact duplicate the parent, except:
 - It gets a new PID
 - It **shares** the same (read-only) text segment
- > Two processes have separate copies of stack, data, and heap
 - Each process can modify variables without affecting other process

Process Creation Example

> Guess the output of the following code:

```
#include <stdio.h>
#include <unistd.h> /* for fork */

void main()
{
    if (fork())
        /* Code executed by parent */
        printf("I am the parent\n");
    else
        /* Code executed by child */
        printf("I am the child\n");
}
```

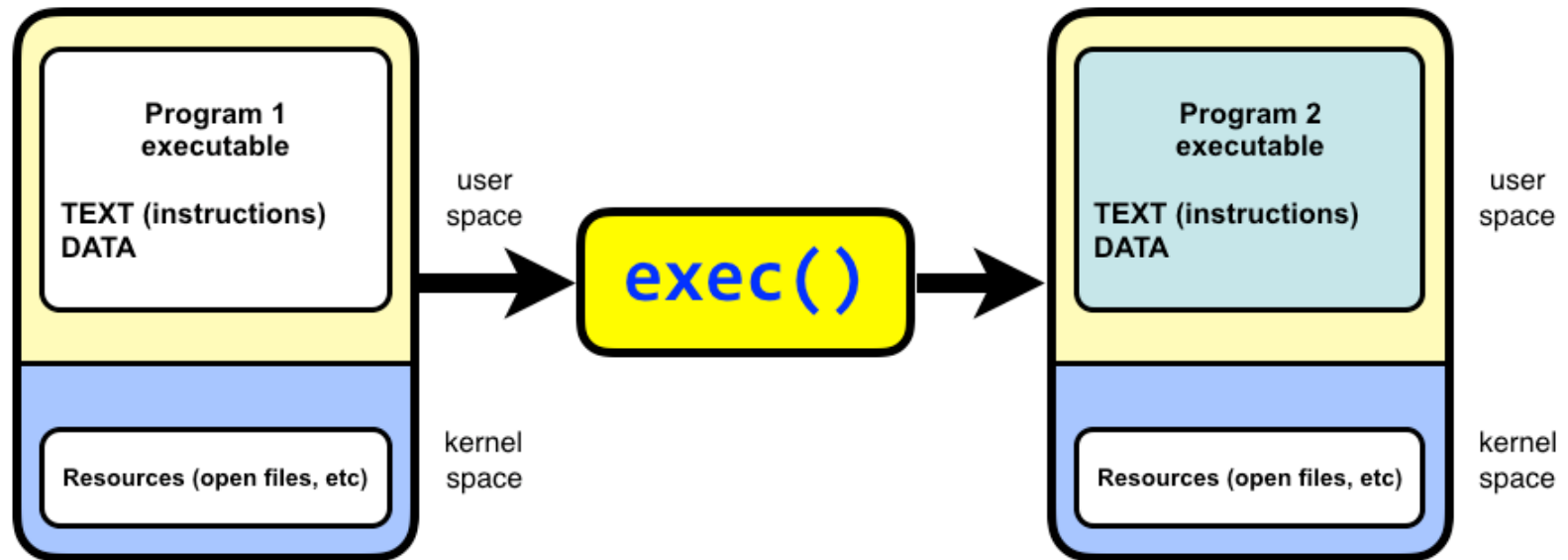
- Concurrent execution
 - ✓ Can't predict execution order of parent and child.
- Shared open files
 - ✓ stdout is the same in both parent and child.

```
I am the parent
$I am the child
□
```

```
I am the child
I am the parent
$□
```

```
I am thI am thee child
parent
$□
```

Program Execution Using exec()



- > The `exec()` family of APIs run an executable file in the context of an existing process, replacing the previous executable.
 - code, data, heap, stack are replaced by those of the new program;
 - but PID, open files, and signal context are kept in the new process image.
 - Never returns on successful execution.

Choosing the Right Version of exec()

> There are 7 variations of `exec()`

- More information: `man 3 exec`

```
#include <unistd.h>

int execve(const char *pathname, char *const argv[],
char *const envp[]);
```

*Absolute/relative path
to the new executable*

*Null-terminated array of pointers
to elements of the environment
list for the new program*

*Null-terminated array of pointers
to the command-line arguments
to be passed to the new program*

Example of Using exec()

> What does the following code do?

```
#include <stdio.h>
#include <unistd.h>

void main()
{
    static char *cmd[]={"ps", "ls", "date", "goof"};
    int i;

    printf("0=ps, 1=ls, 2=date : ");
    scanf("%d",&i);
    execlp(cmd[i], cmd[i], NULL);

    printf("command not found\n");
}
```

Process Termination

- > Process becomes terminated for one of three reasons:
 - Receiving a **signal** whose default action is to terminate (discussed later)
 - Returning from the **main** routine
 - Calling the API functions **exit** (standard C library) or **_exit** (system call)

```
#include <stdlib.h>
void exit(int status)
```

*The lower 8 bits defines **exit status***

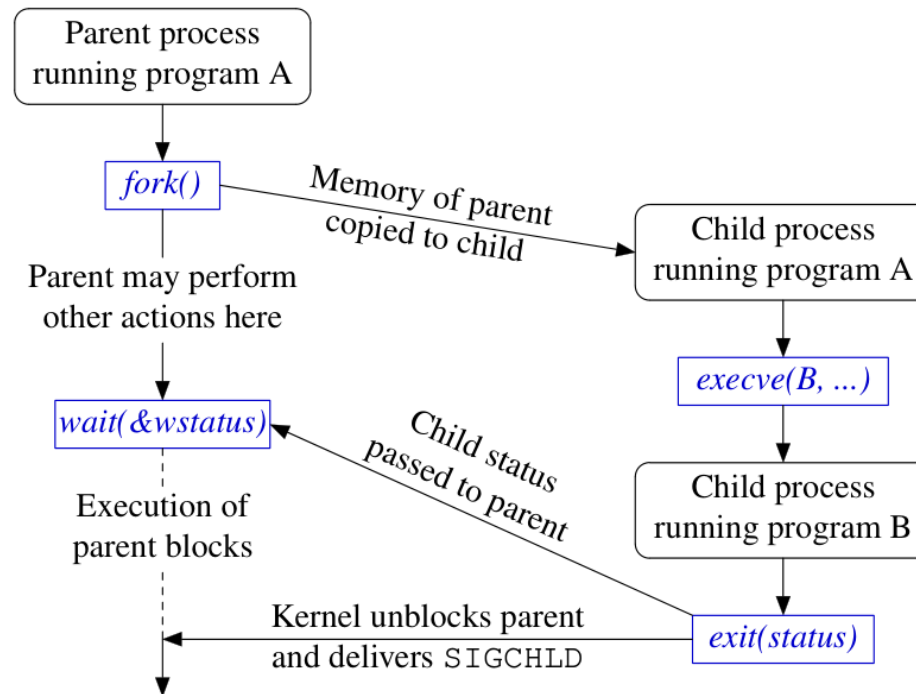
- 0: success
- Nonzero: failure

- > When a child process terminates, it doesn't immediately disappear
 - Enters a **zombie** state - half-dead (stop executing), half alive (as its PCB is still maintained) - until the parent **reaps** the zombie (i.e. picks up the exit status.)
 - The kernel then deletes the zombie and reclaims any remaining resources.
 - If the parent terminates before a child, the **orphan** process will be reaped by init.

Monitoring Child Processes

- > Parent uses `wait()` family of APIs to monitor state change events in child processes, for example:
 - **Termination** (to start reaping the child.)
 - **Stop** (because of a signal) or **continue** (after SIGCONT signal.)
- > Parent can obtain various information about state changes:
 - Exit status of process;
 - What signal stopped or killed process;
 - Whether process produced a core dump before terminating.
- > For historical reasons, there are multiple `wait()` functions
 - More useful variant is: `waitpid()` - waiting for a specific process.
 - For more details, use: `man 2 wait`.

Waiting for Children Processes



> Return value:

- On success: PID of child whose status is being reported.
- On error: -1.

Control Child Process Example

> How to get the child in the previous example to always print first?

```
#include <stdio.h>      /* for printf() */
#include <unistd.h>      /* for fork()  */
#include <sys/wait.h>    /* for wait() */

void main()
{
    int child_pid=fork();
    if (child_pid){
        waitpid(child_pid, &status, 0);
        printf("I am the parent\n");
    }
    else
        printf("I am the child\n");
}
```

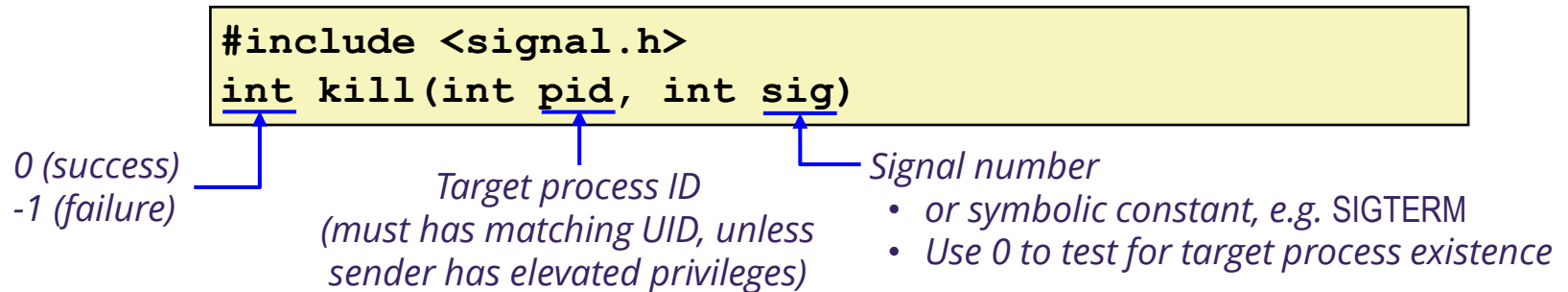
Signals

- > Small messages that notify a process that an event of some type has occurred in the system.
 - A mechanism for inter-process communication, *primarily asynchronous*, and usually generated by the kernel, other processes, or the process itself.
 - The only information in a signal is its ID and the fact that it arrived.
- > When a signal is sent, the kernel ***interrupts*** the target process to
 - invoke the registered *signal handler* if one exists for that signal; otherwise,
 - execute the ***default action*** associated with the signal.

<i>ID</i>	<i>Name (signal.h)</i>	<i>Default Action</i>	<i>Corresponding Event</i>
2	SIGINT	Terminate	Interrupt (e.g., ctrl-c from keyboard)
9	SIGKILL	Terminate	Kill program (cannot override or ignore)
11	SIGSEGV	Terminate & Dump	Segmentation violation
14	SIGALRM	Terminate	Timer signal
17	SIGCHLD	Ignore	Child stopped or terminated

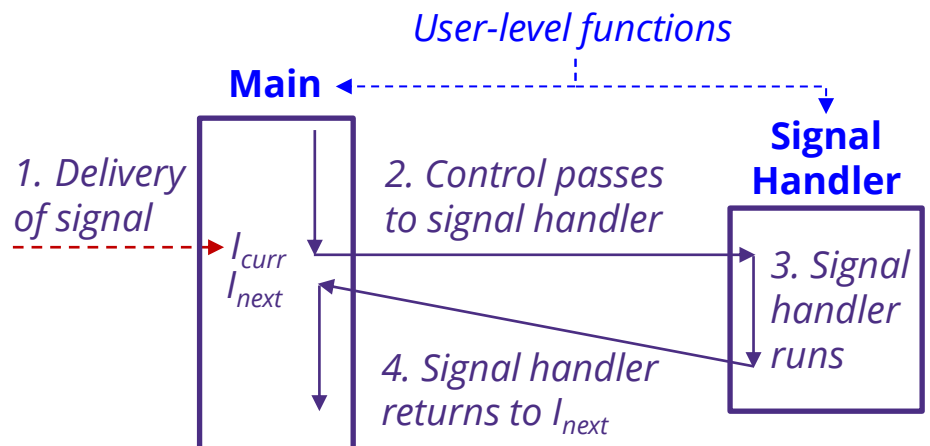
Sending and Receiving Signals

> Send a signal from within a program using the system call **kill()**



> When a signal is received, a process must react in 1 of several ways

- **Ignore** the signal.
- **Terminate** the process.
- **Catch** the signal by executing the so called signal handler.



Establishing a Signal Handlers

- > The easiest way to associate a handler with a signal is using:

```
#include <signal.h>
typedef void (*handler_t)(int)

handler_t signal(int sig, handler_t handler)
```

↑
*Returns a pointer
to the previous
signal handler*

↑
Signal type

↑
Pointer to signal handler function, or one of:

- SIG_IGN: ignore signals
- SIG_DFL: revert to the default action

- > Programming a signal handler is tricky because they are concurrent with main program and share the same global data structures.
- Shared data structures can become corrupted.
 - Signal handler can introduce *race* conditions.

Signal Handler Example

> What happens when you run the following code?

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int n = 0;

void sigint_handler(int sig) {
    printf("Handler catches signal %d\n", sig);
    n = 0;
}

void main() {
    signal(SIGINT, sigint_handler);
    for (int i = 0; i < 10; i++) {
        printf("Main count = %d\n", n++);
        sleep(1);
    }
}
```

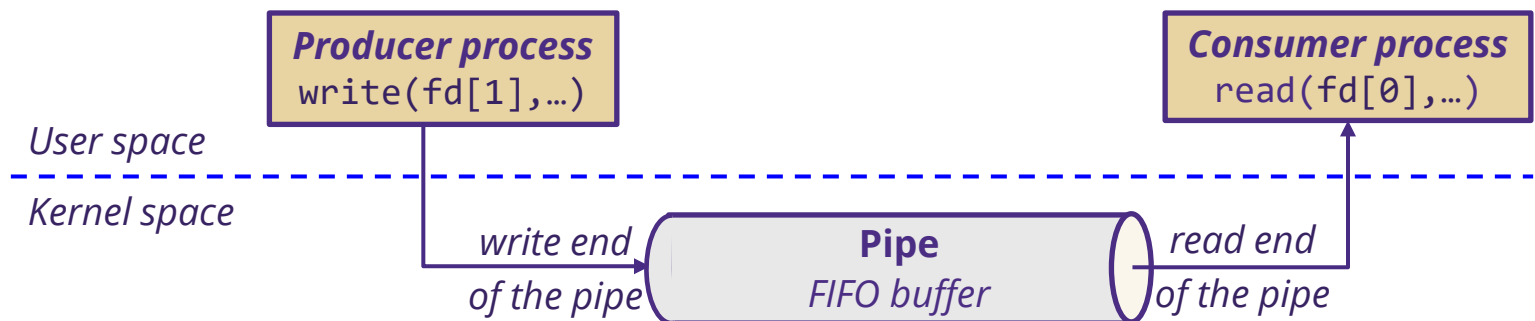
Pipes

> **Unidirectional** interprocess communication mechanism

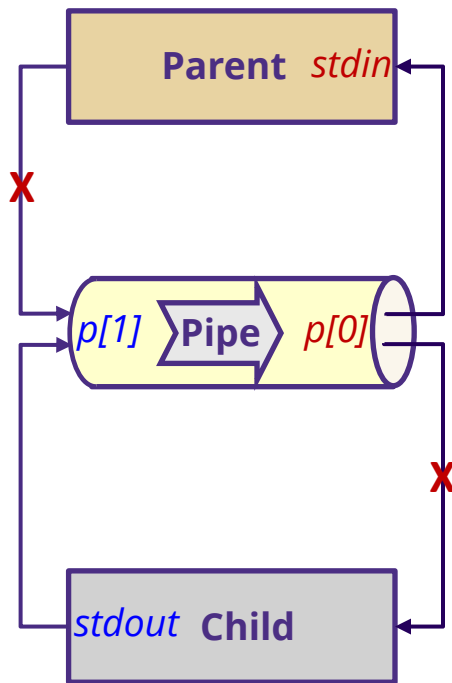
- Provide buffering & loose synchronization between a **producer** & a **consumer**.
 - ✓ Producer blocks when writing if pipe is full
 - ✓ Consumer blocks when reading if pipe is empty
- **Anonymous pipes** can only be used for related processes (e.g. parent & child), whereas **named pipes** can be used for communication between any processes.

> An anonymous pipe is created using `int pipe(int fd[2])`

- Returns 2 file descriptors referring to the read and write ends of the pipe.
- Data can be transferred through the pipe via `write()` and `read()` command.



Connecting Processes to Pipes



```
#include <unistd.h>

void main() {
    int p[2];
    pipe(p);
    if (fork() == 0) {
        /* Child */
        dup2(p[1], 1);
        close(p[0]);
        execlp("ls", "ls", (char *)0);
    }
    /* Parent */
    dup2(p[0], 0);
    close(p[1]);
    execlp("sort", "sort", "-r", (char *)0);
}
```

> Which Shell command is equivalent to the above code?

NEXT LECTURE

[Flipped class] Working with processes

- > Pre-class
 - Study pre-class materials on Canvas
- > In class
 - Reinforcement/enrichment discussion
- > Post class
 - Homework
 - Preparation for Lab 1
 - Consultation (if needed)